MICOM pFBA Stage-3 Infeasibility – Research Findings

MICOM `pfba=True` Stage-3 Infeasibility – Research Findings and Recommended Approach

Compiled: 2026-05-11 **Scope:** All-cohort subject-level MICOM under the `high_fiber` diet (516 subjects, 10 AGORA2 species per community, `cooperative_tradeoff(fraction=0.5)`). **Trigger:** 11 of 516 subjects produced reaction-flux values up to 3.95 million in `Results/subject_level_fba/tables/08_allcohort_subject_reaction_flux_nonzero_long_high_fiber_pfba.csv`, despite every AGORA2 reaction being bounded to ±1000 mmol/gDW/h. The investigation asks (a) whether this is a known MICOM issue, (b) what published cohort studies do, and (c) what approach the project should take.

Executive Summary

The `pfba=True` stage-3 infeasibility is a **known issue acknowledged by the MICOM maintainer** (Christian Diener) in 2020 (micom-dev/micom#11). The maintainer's own recommended workaround is exactly what this project did in script 09: re-run with `pfba=False`. Furthermore, **`pfba=True` is not the published default for at-scale cohort work** — the official `micom.workflows.grow` workflow defaults to `pfba=False`, and neither the original Diener 2020 MICOM paper nor the Quinn-Bohmann 2024 SCFA paper used a pFBA polish in their cohort runs. The project's decision to switch to script 09 is therefore consistent with both the MICOM maintainer's guidance and the published cohort literature.

A separate library-level defect compounds the problem: MICOM's `solve()` function in `micom/solution.py` does not re-validate `community.solver.status` after the pFBA `optimize()` call, so `solution.fluxes` is populated from leftover LP variables when stage 3 fails. This is the root cause of the >1000 garbage values that appeared in script 08's flux table. The defect is documented in MICOM's GitHub issues (#11, #29, #225) but no upstream fix has been merged.

1. The Technical Issue Recap

`community.cooperative_tradeoff(fraction=0.5, fluxes=True, pfba=True)` runs three LP stages internally:

1. **Stage 1** — maximize community growth, producing `G_max`.

2. **Stage 2** — re-solve at `0.5 × G_max` with L2 regularization on per-taxon growth rates, producing per-taxon primals `r_sp`.
3. **Stage 3 (pFBA polish)** — `add_pfba_objective` pins each `r_sp` as a near-equality constraint via `obj.lb = (1 − rtol) · r_sp − atol` (defaults `atol = rtol = 1e-6`), then minimizes Σ|flux|.

For 11 of 516 subjects, stage 2 sat on a degenerate LP vertex (objective values from 66,408 up to 43,949,989, against a cohort norm below 100). Stage 3's near-equality pinning then made the joint constraints unsatisfiable within solver tolerance, returning `infeasible`. MICOM's `solve()` proceeded to construct `CommunitySolution` from leftover LP variable values regardless, yielding fluxes that violated the SBML ±1000 reaction bounds.

The 11 affected subjects (audit-stable list): `MBS1232, MBS1529, MBS1262, MBS1255, MBS1539, MHS276, MBS1535, MBS1438, CON079, MBS1576, MBS1439`

Under script 09 (`pfba=False`), all 11 solve optimally with community growth in the 0.24–0.65 range, in line with the rest of the cohort.

2. Section A – Is This a Known Issue?

Yes, and the maintainer recommends the same fix the project adopted.

MICOM Issue #11 – `cooperative_tradeoff` with `fluxes=True` not feasible

- Source: micom-dev/micom#11 (closed, labelled `numeric` and `done`)
- Reporter (`ychoi6`) hit `OptimizationError: crossover could not converge (status = infeasible)` from `cooperative_tradeoff(..., fluxes=True, pfba=True)` on a roughly 50-member community, while `pfba=False` worked.
- Christian Diener (MICOM author) replied:

> "Just to confirm: that only occurs with `pfba=True`, right? This is related to numerical accuracy issues when changing from QP to LP problems and back. ... One workaround is to set the minimal medium and resolving with `pfba=False` — this is faster and usually still gives a parsimonious solution."

- Reporter confirmed `pfba=False` resolved it. Diener later noted: "PFBA now uses the solver tolerance and allows setting tolerances as well which should help work around that."

MICOM Issue #29 – fraction variation influences `status`

- Source: micom-dev/micom#29

- Diener clarifies that `cooperative_tradeoff` returning the `numeric` status typically reflects "tolerance-relaxation issues in large/unstable models," and that "models that large can easily become unstable, mostly when either including taxa in very small abundances (relative abundances < 1e-4) or when having medium components with very small flux bounds."
- Recommends loosening tolerance from the default `1e-6` to `1e-5` or `1e-4`.

micom-dev/paper Issue #5 – "All numerical optimizations failed"

- Source: micom-dev/paper#5
- Diener's diagnostic recipe: load the failing model and run `cooperative_tradeoff(fraction=0.5)` to see the underlying solver error. The reporter's root cause turned out to be a stale diet file.

MICOM Issue #225 – "Exchange metabolites showing very high flux"

- Source: micom-dev/micom#225
- No clean resolution, but confirms that bound-violating fluxes have been observed by other MICOM users.

Upstream library defect – `solve()` does not re-check status after pFBA

- Source file: `micom/solution.py` on `main`
- The `solve()` function (the one called inside `cooperative_tradeoff`) checks the solver status once, *before* the pFBA `optimize()`. After `add_pfba_objective(...)` and the second `optimize()`, no second status check happens. `CommunitySolution(community)` then reads `solver.primal_values` directly, regardless of whether the post-pFBA LP is feasible.
- No PR has been merged that adds a status gate after the pFBA stage. This is the upstream-level reason that script 08 (in its original commit `371cf94`) silently exported >1000 fluxes.

3. Section B – What Do Published MICOM Papers Actually Use?

Diener et al. 2020 – original MICOM paper

- Citation: Diener C, Gibbons SM, Resendis-Antonio O. *MICOM: Metagenome-Scale Modeling To Infer Metabolic Interactions in the Gut Microbiota*. mSystems 5(1) (2020). DOI: 10.1128/msystems.00606-19. Open-access copy: PMC6977071.
- Tradeoff fraction: **0.5** ("the best agreement with the observed replication rates... was observed at a trade-off value of 0.5").
- pFBA polish: **not used.** The paper instead relies on the L2 cooperative-tradeoff QP plus an explicit crossover step "where we took the solution of the numerically ill-conditioned quadratic interior point method as a candidate solution set."
- No per-subject infeasibility QC reported.

Quinn-Bohmann et al. 2024 – personalized SCFA / MCMM paper

- Open-access copies: PMC11841136 and PMC10002715.
- Tradeoff fraction: **0.7**, chosen as "the highest parameter value … that allowed most (>90%) of taxa to grow."
- pFBA polish: **not mentioned.** The method is described as "cooperative tradeoff flux balance analysis (ctFBA)" without a pFBA polish step.
- QC gate: every model had to grow at minimum community growth ≥ 0.3 h^{-1}.

vrmarcelino/MetaModels reference script

- Source: `MICOM_coop_tradeoff.py`
- Calls `com.cooperative_tradeoff(fraction=0.5, fluxes=True, pfba=False)` with the inline comment *"uses the 'classic' FBA instead of the parsimonious FBA."* The `pfba=True` line is left commented out.

Gibbons Lab `isb_course_2020` teaching notebook

- Repository: Gibbons-Lab/isb_course_2020.
- Uses `cooperative_tradeoff` at the tutorial defaults; pFBA is not enabled.

4. Section C – What Does MICOM's Documentation Recommend?

Method-level docstring

- Reference: `micom.community` API docs.
- The `cooperative_tradeoff` docstring describes pFBA as "highly recommended."
- Default value of the `pfba` argument is `False`.

`micom.workflows.grow` – official at-scale entry point

- Source: `grow.py` on `main`.
- The `strategy` argument defaults to `"minimal imports"`, which internally calls `cooperative_tradeoff(fluxes=False, pfba=False)`. Only the explicitly opt-in `strategy="pFBA"` sets `pfba=True`.
- Tradeoff guidance: *"Tradeoff values of 0.5 for metagenomics data and 0.3 for 16S data seem to work well."*

`micom` Logic page

- Reference: MICOM Logic documentation.
- Documents the QP→crossover strategy for the cooperative tradeoff QP step. pFBA is not described as part of the documented core algorithm.

Net effect: The docstring's "highly recommended" wording is technically accurate for individual `cooperative_tradeoff` calls but is contradicted by the at-scale workflow's own defaults. Cohort-style work follows the workflow defaults; per-community work can opt in to pFBA case by case.

5. Section D – Documented Mitigations Short of Dropping pFBA

If at any future point the project decides to retry `pfba=True` for a subset (for example, only the 11 affected subjects), the documented mitigations are:

- **Loosen `atol` / `rtol`** from the default `1e-6` to `1e-5` or `1e-4`. Diener in Issue #29: "still a fairly accurate solution up to a tolerance of 1e-5 or 1e-4." Exposed since MICOM v0.22.1.
- **Enable `presolve=True`** in the workflow path for additional numerical stability.
- **Drop taxa with relative abundance below `1e-4`**, per Diener's diagnosis in Issue #29 (very low abundance is one of the two main destabilizers).
- **Add a `solution.status == "optimal"` gate** before exporting fluxes. This is currently the script-side defense and is what `Scripts/modelling/09_micom_allcohort_subject_level_high_fiber_no_pfba.py` already does at line 506 — and what the current revision of `Scripts/modelling/08_micom_allcohort_subject_level_high_fiber_pfba.py` already does at line 498 (added in commit `9b4ede4`).
- **No published wrapper** automatically falls back from `pfba=True` to `pfba=False` on per-subject failure. The "fall back to no-pFBA" choice the project made is therefore a reasonable original mitigation rather than an established pattern.

6. Section E – Recommended Approach for This Project

Primary action: keep `pfba=False` as the canonical flux branch

The script 09 outputs (`Results/subject_level_fba/tables/09_…no_pfba.csv`) are already the canonical flux/growth tables for downstream PCA and dimensionality reduction. Nothing about the upstream evidence calls that decision into question. On the contrary:

- It matches the MICOM maintainer's own recommended workaround for the exact failure mode encountered (Issue #11).
- It matches the official at-scale workflow's default (`strategy="minimal imports"` in `grow.py`).
- It matches both Diener 2020 and Quinn-Bohmann 2024 cohort practice.

Defensible scripting standard

Whether or not the project re-enables pFBA in the future, the script-level standard is non-negotiable:

- Always gate `solution.fluxes` and `solution.growth_rate` writes on `solver.status == "optimal"`.
- Treat the combination `solver_status == "infeasible"` and `community_growth_rate == 0.0` as the bug signature, not a biological zero.
- Keep `Scripts/modelling/08_…py` outputs on disk as audit-only; do not consume them downstream. (Already documented in `MD/micom_practices.md` Practice 10.)

How to phrase the supervisor conversation

A literature-supported single-sentence answer to "why didn't you use pFBA":

> The MICOM maintainer's own recommendation in issue #11 is to use `pfba=False` when stage-3 polishing is infeasible, and that is also what the official `micom.workflows.grow` workflow defaults to and what the Diener 2020 and Quinn-Bohmann 2024 cohort papers used.

If the supervisor still wants pFBA flux for those 11 subjects specifically, the honest answer is that the documented mitigations (relax `atol` / `rtol`, `presolve=True`, drop sub-1e-4 taxa) may help but are not guaranteed to recover the 11, and any successful pFBA values would still need a clear flag column distinguishing them from the no-pFBA mainline.

Trade-off to acknowledge openly

Without pFBA, the LP returns *some* optimal vertex rather than the parsimonious one. A reaction value sitting at exactly ±1000 in the script 09 flux table is saturated against the SBML reaction bound, not a measure of metabolic intensity. This caveat is already documented in `MD/micom_practices.md` Practice 10 and should be applied during interpretation.

7. Reference URLs

- MICOM Issue #11 (the canonical match): https://github.com/micom-dev/micom/issues/11
- MICOM Issue #29 (numerical/status guidance): https://github.com/micom-dev/micom/issues/29
- micom-dev/paper Issue #5 (cohort failure handling): https://github.com/micom-dev/paper/issues/5
- MICOM Issue #225 (high exchange flux): https://github.com/micom-dev/micom/issues/225
- MICOM `solution.py` source (the unfixed `solve()` defect): https://github.com/micom-dev/micom/blob/main/micom/solution.py
- MICOM `grow` workflow defaults (`pfba=False`): https://github.com/micom-dev/micom/blob/main/micom/workflows/grow.py
- MICOM `cooperative_tradeoff` API docs: https://micom-dev.github.io/micom/autoapi/micom/community/index.html
- MICOM Logic page: https://micom-dev.github.io/micom/logic.html
- MICOM NEWS / changelog: https://github.com/micom-dev/micom/blob/main/NEWS.md
- Diener 2020 MICOM paper: https://pmc.ncbi.nlm.nih.gov/articles/PMC6977071/
- Quinn-Bohmann 2024 SCFA paper: https://pmc.ncbi.nlm.nih.gov/articles/PMC11841136/
- Marcelino MetaModels reference script: https://github.com/vrmarcelino/MetaModels/blob/main/MICOM_coop_tradeoff.py

8. Internal Project Cross-References

- `MD/micom_practices.md` Practice 10 — full project-internal write-up of the pFBA stage-3 infeasibility and the `pfba=False` decision.
- `Scripts/modelling/08_micom_allcohort_subject_level_high_fiber_pfba.py` — original pFBA branch; current code has the status gate but those 11 subjects will still appear as `infeasible`.
- `Scripts/modelling/09_micom_allcohort_subject_level_high_fiber_no_pfba.py` — canonical no-pFBA branch; produces clean flux outputs for all 516 subjects.
- `Results/subject_level_fba/tables/08_…_pfba.csv` files — kept as audit-only artifacts, not for downstream use.
- `Results/subject_level_fba/tables/09_…_no_pfba.csv` files — canonical flux/growth tables.